


**Laboratorium**
**Robotika dan Sistem Cerdas**
**Departemen Teknik Komputer**
***Institut Teknologi Sepuluh Nopember***

# Laporan Sementara
# Praktikum Sistem Mikroprosesor dan Sistem Tertanam

## Pengantar Mikroprosesor

Anak Agung Ngurah Agung Kresna Ananta - 5024241085

2026

# 1 Pendahuluan

## 1.1 Latar Belakang

Mikrokontroler ada di hampir semua perangkat elektronik, dari peralatan rumah tangga hingga *node* IoT. Mikrokontroler adalah satu chip yang berisi CPU, memori, dan periferal I/O. Sebelum masuk ke topik seperti protokol komunikasi, RTOS, atau konektivitas nirkabel, praktikan perlu bisa memprogram mikrokontroler dan mengakses periferal paling dasarnya, yaitu GPIO (*General Purpose Input/Output*).

Praktikum ini memakai dua board yang cukup berbeda. STM32 Blackpill berbasis ARM Cortex-M dan di-*flash* melalui SWD, sedangkan ESP32 berbasis Xtensa LX6 dan di-*flash* melalui USB-to-Serial. ESP32 juga diprogram dengan dua *framework*, yaitu Arduino yang berabstraksi tinggi dan ESP-IDF yang bersifat *native*. Di sisi perangkat keras, ada tiga masalah yang tidak muncul pada pemrograman perangkat lunak biasa: pin input yang *floating*, sinyal tombol yang mengalami *bouncing*, dan besaran analog yang harus dibaca lewat ADC. Praktikum ini melatih praktikan menyiapkan PlatformIO, melakukan *flashing* ke kedua board, dan menggunakan GPIO dalam tiga mode: *digital output*, *digital input*, dan *analog input*.

## 1.2 Dasar Teori

### 1.2.1 PlatformIO sebagai Lingkungan Pengembangan

PlatformIO adalah ekosistem pengembangan *embedded* lintas platform yang dipasang sebagai ekstensi Visual Studio Code. PlatformIO mengunduh dan mengatur *toolchain* (kompiler, *linker*), *framework*, *library*, dan *uploader* secara otomatis sesuai isi berkas `platformio.ini`. Berkas ini memuat parameter `platform`, `board`, `framework`, dan `upload_protocol`. Untuk memprogram ESP32 dengan ESP-IDF alih-alih Arduino, misalnya, cukup ubah nilai `framework`. Sebuah *project* terdiri atas folder `src/` (kode utama), `include/` (*header*), dan `lib/` (*library* tambahan).

### 1.2.2 STM32 Blackpill dan Framework Arduino (STM32duino)

Blackpill (WeAct STM32F401CCU6) adalah *development board* berbasis ARM Cortex-M4 32-bit dengan FPU, *clock* hingga 84 MHz, Flash 256 KB, dan SRAM 64 KB. LED *onboard* terhubung ke pin PC13 dan bersifat *active-low*, sedangkan tombol *user* terhubung ke PA0. Board ini tidak memiliki *programmer onboard*, jadi *flashing* dilakukan dengan ST-Link melalui *Serial Wire Debug* (SWD). SWD hanya memakai dua jalur, yaitu SWDIO (data dua arah) dan SWCLK (*clock*). Lewat *Debug Access Port* (DAP) pada inti Cortex-M, ST-Link dapat menulis Flash secara langsung dan juga melakukan *debugging* (*breakpoint*, *step*, inspeksi register). Pada praktikum ini STM32 diprogram dengan STM32duino, yaitu lapisan Arduino yang dibangun di atas HAL STMicroelectronics. Dengan STM32duino, fungsi seperti `pinMode()`, `digitalWrite()`, dan `delay()` tetap bisa dipakai, dan pin dapat ditulis sesuai label port fisiknya (misalnya `PC13`).

### 1.2.3 ESP32 dan Framework ESP-IDF

ESP32 adalah SoC dengan dua inti Xtensa LX6 (hingga 240 MHz), SRAM 520 KB, serta WiFi 802.11 b/g/n dan Bluetooth terintegrasi. Board ESP32 DevKit v1 memiliki chip *bridge* USB-to-UART (CP2102/CH340)

dan rangkaian *auto-reset*, jadi board dapat di-*flash* langsung lewat kabel USB menggunakan *bootloader* serial di ROM. LED *onboard* umumnya terhubung ke GPIO2. ESP-IDF (*Espressif IoT Development Framework*) adalah *framework* resmi Espressif yang dibangun di atas FreeRTOS. Program dimulai dari fungsi `app_main()`, yang dijalankan sebagai *task* setelah *scheduler* aktif. Fungsi `vTaskDelay(pdMS_TO_TICKS(ms))` memindahkan *task* ke status *Blocked* selama sejumlah *tick*. Selama itu CPU bebas menjalankan *task* lain, berbeda dengan *busy-waiting* yang menghabiskan siklus CPU hanya untuk menunggu.

### 1.2.4 Digital Input, Kondisi *Floating*, dan Resistor Pull-up/Pull-down

Pin input digital CMOS memiliki impedansi sangat tinggi. Jika tidak terhubung ke level tegangan tertentu, pin berada dalam kondisi *floating*: *noise* dan kapasitansi parasit membuatnya terbaca HIGH atau LOW secara acak. Resistor *pull-up* (pin ke VCC) atau *pull-down* (pin ke GND) memberi pin level *default* yang pasti. Nilai 10 k$\Omega$ umum dipakai karena cukup kuat menahan *noise*, sementara arus saat tombol ditekan hanya $I = 3{,}3\ \mathrm{V}/10\ \mathrm{k\Omega} = 0{,}33\ \mathrm{mA}$. ESP32 juga memiliki resistor *pull* internal (sekitar 45 k$\Omega$) yang diaktifkan dengan `INPUT_PULLUP` dan `INPUT_PULLDOWN`. Pin GPIO34–39 adalah pengecualian karena bersifat *input-only* dan tidak memiliki resistor *pull* internal.

### 1.2.5 *Bouncing* dan *Debouncing*

Kontak logam pada tombol mekanik bersifat elastis. Saat tombol ditekan atau dilepas, kontak memantul beberapa kali sebelum stabil dan menghasilkan deretan transisi HIGH–LOW selama ratusan mikrodetik hingga beberapa milidetik. Mikrokontroler membaca pin dalam orde mikrodetik, jadi setiap pantulan bisa terhitung sebagai *event* terpisah. *Debouncing* berbasis waktu mencatat waktu perubahan terakhir dengan `millis()` dan baru menerima status baru jika sinyal stabil lebih lama dari $T_{\mathrm{debounce}}$ (umumnya 20–50 ms). Semakin besar nilainya, pembacaan semakin bersih, tetapi tombol semakin lambat merespons.

### 1.2.6 ADC sebagai Mode Akses GPIO Ketiga

Selain sebagai *digital output* dan *digital input*, GPIO dapat dipakai sebagai *analog input* melalui ADC (*Analog-to-Digital Converter*). ESP32 memiliki dua ADC bertipe SAR (*Successive Approximation Register*), yaitu ADC1 (GPIO32–39) dan ADC2. ADC2 dipakai oleh *driver* WiFi dan tidak bisa digunakan saat WiFi aktif, jadi ADC1 lebih disarankan. Dengan resolusi *default* $N = 12$ bit, nilai digital $D$ dan tegangan hasil rekonstruksinya adalah

$$D = \mathrm{round}\left(\frac{V_{in}}{V_{ref}}\left(2^N - 1\right)\right), \qquad V \approx D \cdot \frac{3{,}3}{4095}. \tag{1}$$

Pada *framework* Arduino, atenuasi *default* 11 dB membuat rentang masukan mendekati 0–3,3 V, tetapi respons ADC ESP32 nonlinier di ujung rentang (mendekati 0 V dan di atas sekitar 2,5 V). Pada percobaan, LDR dirangkai sebagai pembagi tegangan dengan $R_1 = 10\ \mathrm{k\Omega}$:

$$V_o = V_{in}\,\frac{R_1}{R_{LDR} + R_1}. \tag{2}$$

Semakin terang cahaya, $R_{LDR}$ semakin kecil, sehingga $V_o$ dan nilai ADC semakin besar.

## 2 Tugas Pendahuluan

1. **Perbedaan framework Arduino dan ESP-IDF.** Arduino adalah lapisan abstraksi tingkat tinggi dengan struktur `setup()` dan `loop()` serta API yang sama di berbagai board. ESP-IDF adalah *framework native* Espressif berbasis FreeRTOS. Program ESP-IDF dimulai dari `app_main()` dan dapat mengakses *driver* periferal serta fitur RTOS secara langsung. Pada Arduino-ESP32 pun, `loop()` sebenarnya berjalan di dalam sebuah *task* FreeRTOS, hanya saja disembunyikan dari pengguna.

**Tabel 1:** Perbandingan framework Arduino dan ESP-IDF

| **Aspek** | **Arduino** | **ESP-IDF** |
|---|---|---|
| Titik masuk | `setup()`, `loop()` | `app_main()` |
| Abstraksi | Tinggi, API seragam | Rendah-menengah, spesifik ESP32 |
| RTOS | Tersembunyi/terbatas | Langsung (*task*, *queue*, semaphore) |
| Kelebihan | Mudah dipelajari, *library* melimpah, kode portabel, cocok untuk prototipe cepat | Kontrol penuh (WiFi, *power management*, *partition*), performa dan determinisme lebih baik, didukung resmi |
| Kekurangan | Kontrol periferal terbatas, ada *overhead* abstraksi, kurang cocok untuk produk kompleks | Lebih sulit dipelajari, kode lebih panjang, tidak portabel ke board lain |

2. **ST-Link pada Blackpill vs USB pada ESP32.** ESP32 DevKit memiliki chip *bridge* USB-to-Serial (CP2102/CH340) yang menerjemahkan sinyal USB dari PC menjadi UART untuk UART0 ESP32. Rangkaian *auto-reset* memakai sinyal DTR/RTS untuk mengendalikan pin EN dan GPIO0. Dengan cara ini `esptool` dapat memaksa ESP32 masuk ke *bootloader* serial di ROM dan menulis Flash tanpa pengguna perlu menekan tombol apa pun. Blackpill tidak memiliki chip *bridge* tersebut. Port USB-nya terhubung langsung ke periferal USB OTG milik STM32, yang secara *default* tidak menjalankan *bootloader* serial. Karena itu, cara paling andal adalah ST-Link melalui SWD, yang mengakses inti dan memori Flash secara langsung lewat *debug port* dan sekaligus bisa dipakai untuk *debugging*. STM32F401 sebenarnya juga dapat di-*flash* via USB DFU dengan menahan tombol BOOT0, tetapi cara ini manual dan tidak mendukung *debugging*.

3. **Kondisi logika pull-up dan pull-down.** Pada *pull-up*, pin ditarik ke VCC dan terbaca HIGH saat tombol dilepas. Saat ditekan, tombol menghubungkan pin ke GND dengan hambatan yang jauh lebih kecil daripada resistor, jadi pin terbaca LOW. Konfigurasi ini disebut *active-low*. *Pull-down* bekerja sebaliknya (*active-high*).

**Tabel 2:** Kondisi logika pin pada konfigurasi pull-up dan pull-down

| **Konfigurasi** | **Tidak ditekan** | **Ditekan** | **Sifat** |
|---|---|---|---|
| Pull-up (ke VCC, tombol ke GND) | HIGH | LOW | *active-low* |
| Pull-down (ke GND, tombol ke VCC) | LOW | HIGH | *active-high* |

4. **Fenomena bouncing dan cara mengatasinya.** Saat kontak tombol bertemu atau berpisah, pegas dan massa kontak bergetar. Sambungan listrik pun terbuka dan tertutup berulang kali

selama beberapa milidetik sebelum stabil. Beberapa cara untuk mengatasinya:

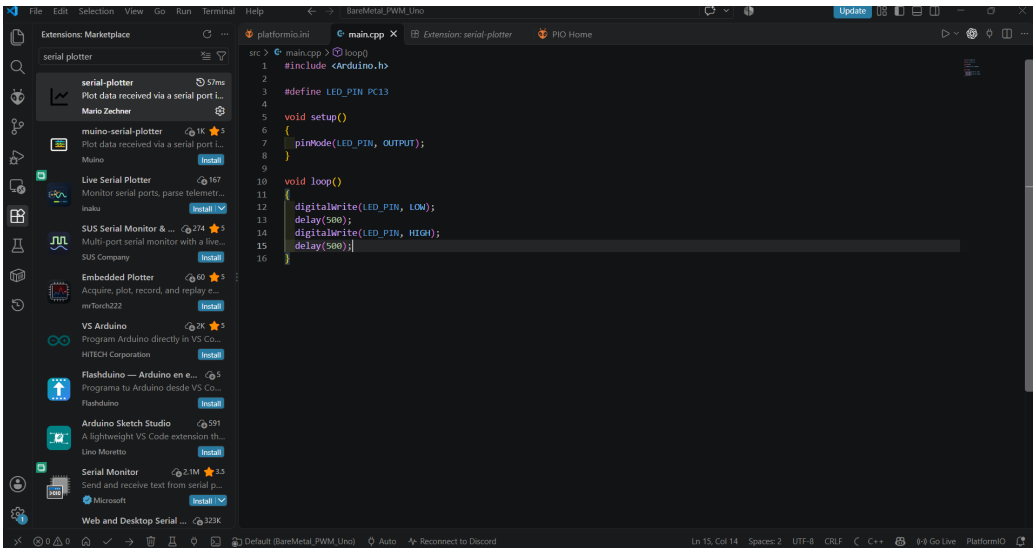
- *Hardware debouncing*: filter *low-pass* RC meredam transisi cepat, lalu *Schmitt trigger* (misalnya 74HC14) merapikan tepi sinyalnya. Untuk saklar SPDT dapat digunakan SR *latch*.
- *Software debouncing* berbasis waktu: catat `millis()` setiap kali sinyal berubah, dan perbarui status hanya jika sinyal stabil lebih lama dari $T_{\text{debounce}}$, seperti pada Percobaan 4. Metode ini *non-blocking*, berbeda dengan `delay()` biasa.
- *Software debouncing* berbasis *sampling*/*counter*: baca pin secara periodik (misalnya lewat *timer interrupt*) dan terima status baru setelah $n$ sampel berturut-turut bernilai sama.

5. **Cara kerja ADC dan pengaruh resolusi.** ADC mengubah sinyal analog dalam tiga tahap. Pada tahap *sampling* dan *hold*, tegangan masukan dicuplik lalu ditahan pada kapasitor. Tegangan itu kemudian dikuantisasi ke salah satu dari $2^N$ level diskrit, lalu levelnya dikodekan (*encoding*) sebagai bilangan biner. ADC tipe SAR seperti pada ESP32 bekerja dengan pencarian biner: $V_{in}$ dibandingkan dengan keluaran DAC internal bit demi bit dari MSB ke LSB, jadi satu konversi selesai dalam $N$ langkah. Resolusi menentukan ukuran langkah terkecil:

$$\text{LSB} = \frac{V_{ref}}{2^N}. \tag{3}$$

Dengan $V_{ref} = 3{,}3$ V, LSB ADC 8-bit sekitar $12{,}9$ mV, sedangkan LSB ADC 12-bit sekitar $0{,}81$ mV. Galat kuantisasi maksimum adalah $\pm\frac{1}{2}$ LSB, jadi semakin besar $N$, semakin presisi pembacaannya. Presisi efektif tetap dibatasi oleh *noise*, nonlinieritas, dan akurasi tegangan referensi, yang dinyatakan sebagai ENOB (*Effective Number of Bits*). ENOB ADC ESP32 lebih rendah dari 12 bit, sehingga hasil pembacaan perlu dikalibrasi atau diambil rata-rata dari beberapa sampel.

6. **Set-up perangkat lunak.** Visual Studio Code dan ekstensi PlatformIO IDE sudah terinstal. Platform *Espressif 32* (*framework* ESP-IDF dan Arduino) dan *ST STM32* (*framework* Arduino/STM32duino) sudah ditambahkan melalui PlatformIO Home. Ekstensi *Serial Plotter* juga sudah dipasang untuk menampilkan data serial dalam bentuk grafik, misalnya hasil pembacaan ADC. *Driver* ST-Link dan CP2102/CH340 sudah terinstal, dan kedua board terdeteksi oleh sistem operasi. Tangkapan layarnya ditunjukkan pada Gambar 1.




**Gambar 1:** Screenshoot VSCode dengan PlatformIO serta ekstensi Serial Plotter yang telah terpasang